Principles for Writing Clean Code

A clean code is one that is easily understandable and maintainable. It's an act of caring for your future self and your collaborators, as you will likely be the one who has to revisit and modify your code later on. Investing time in writing clean code pays off in the long run by saving you from the frustration of deciphering a messy codebase.

The principles mentioned here are foundational clean code practices, some of which reflect core ideas from the SOLID principles, especially the Single Responsibility Principle, and align with common design pattern concepts.

1. Use Meaningful and Descriptive Naming

Variables, functions, classes, and other entities should have names that clearly communicate their purpose, without requiring additional comments. The goal is to make the code self-documenting.

- **Be explicit and clear:** A well-chosen name helps you and others understand what the entity is for without having to read through its implementation.

  Bad example: `l = 100` (What does 'l' mean?)

  Better exapmle `aruco_length=100` (This is much clearer.)

- **Avoid overly long or cryptic names:** Find a balance between being descriptive and concise. A name should be just long enough to convey its meaning accurately.

- **Keep the same Naming style:** Use a consistent naming convention throughout your codebase so that variables, functions, classes, and constants follow the same style rules. This makes the code easier to read, predict, and maintain.

  For example:

  - If you choose **snake_case** for variables and functions, stick with it everywhere instead of mixing it with camelCase.
  - Use **PascalCase** for class names if that's your chosen standard.

  More information for Naming Conventions in Python: link

2. Avoid Magic Numbers

A "magic number" is a numeric literal (e.g., 3.14, 100, 7) that appears directly in the code without any explanation of what it represents. This makes the code difficult to read and modify.

- **What to do instead:** Use a named constant to represent the value. This gives the number a clear, descriptive name, making the code more readable and easier to change later.

  Bad example:
  ```
  pixel_cm_ratio = detected_aruco_perimeter/40
  ```
  -40 can be stored in a more meaningful variable as bellow.

  Better Example:
  ```
  aruco_perimeter_in_cm = 4*aruco_length
  ```

3. Write Meaningful Comments

The best code is self-explanatory and requires few comments. However, there are times when a comment is necessary and helpful.

- **Explain "Why," not "What":** Good comments explain the rationale behind a decision or a complex logic, not simply what the code is doing. The code should already explain the "what."
- **Use them for clarifying complex parts:** If a piece of code is genuinely tricky or uses a non-obvious algorithm, a comment can be invaluable for future readers.

4. Write Small and Focused Functions/Methods

The core idea is that a function should have a **single responsibility principle (SPR)**. When a function does "one thing and one thing well," its purpose is clear. This makes your code more readable, testable, and easier to maintain.

Here are some guidelines to help you decide:

1. If the code has a clear, well-defined purpose that can be given a descriptive name. Bad example:

```python
# ... a bunch of code
# ...
result = [item for item in items if item.status == "active"]
# ... a bunch more code
```

Good example:

```python
def get_active_items(items):
    return [item for item in items if item.status == "active"]

# ... a bunch of code
# ...
result = get_active_items(items)
# ... a bunch more code
```

Even if `get_active_items` is only used once, its name immediately clarifies the intent of that line of code, improving readability.

2. When a block of code performs a complex or non-obvious operation. If you find yourself needing to write a comment to explain a few lines of code, that's often a strong signal that those lines should be extracted into a function with a descriptive name. The function's name becomes the comment.

3. When you are repeating yourself (The DRY principle: Don't Repeat Yourself). This is the most obvious case. If you have the same or similar code in multiple places, putting it into a function is a must.

4. When the code would make your current function too long or complex. If a function starts to grow and you can identify distinct sub-tasks within it, refactor those sub-tasks into their own functions. This makes your main function a high-level "story" of the process, which is much easier to read.